

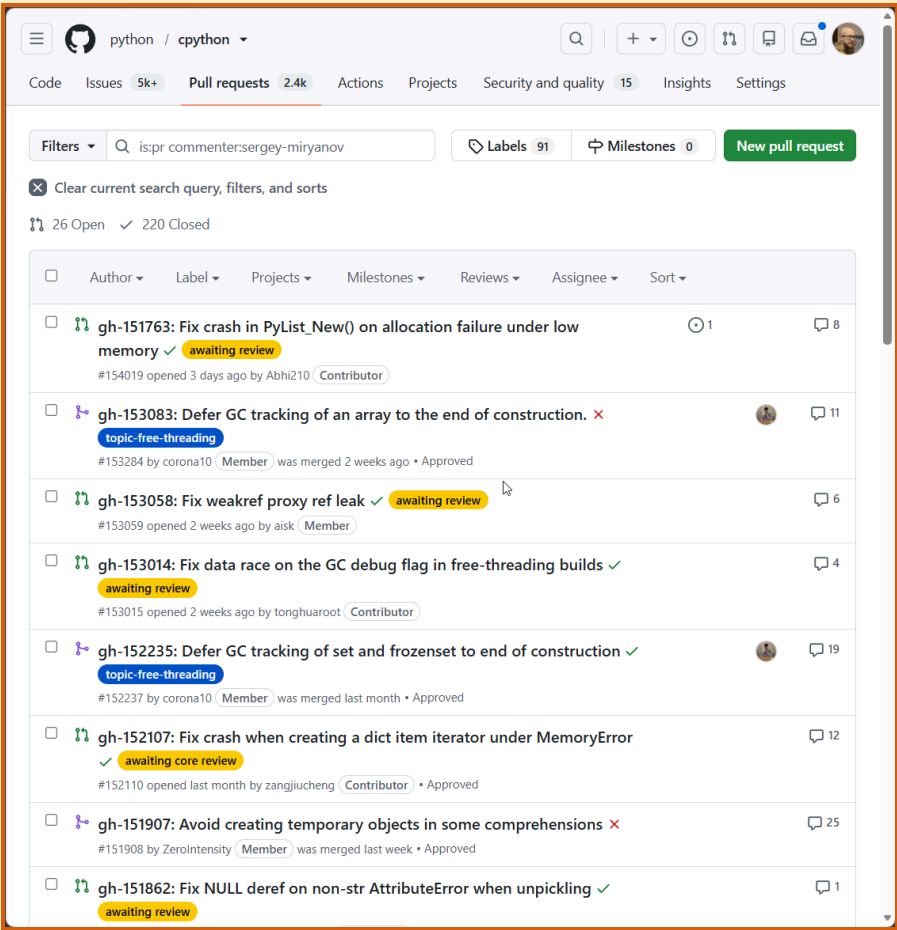
Мониторинг событий сборщика мусора в Python

Вчера, сегодня, завтра?

Мирянов Сергей

PyCon Russia 2026

CPYTHON КОНТРИБЬЮТОР



Python Developer's Guide

This guide is a comprehensive resource for contributing to [Python](#) – for both new and experienced contributors. It is [maintained](#) by the same community that maintains Python. We welcome your contributions!

Start with the area that best matches what you want to do. If you still have questions after reviewing the material in this guide, then the [Core Python Mentorship](#) group is available to help guide new contributors through the process.

Documentation	Code	Triage
Helping with documentation Getting started Style guide reStructuredText primer Translating Helping with the Developer's Guide Guidelines for using AI tools	Setup and building Where to get help Lifecycle of a pull request Running and writing tests Fixing "easy" issues (and beyond) Following Python's development Git bootcamp and cheat sheet Development cycle Guidelines for using AI tools	Issue tracker Triageing an issue Helping triage issues Experts index GitHub labels GitHub issues for BPO users Triage Team

We **recommend** that sections of this guide be read as needed. You can stop where you feel comfortable and begin contributing immediately without reading and understanding everything. If you do choose to skip around within the guide, be aware that some sections build on each other, so you may need to backtrack for missing concepts or terminology.

О ЧЁМ БУДЕМ ГОВОРИТЬ

- Управление памятью
- Сборка мусора (GC, Garbage Collector, Garbage Collection)
- События сборки мусора

ПРОБЛЕМА

Проблема ●

Инструменты ○

Новое API ○

gcmop ○

Perfetto ○

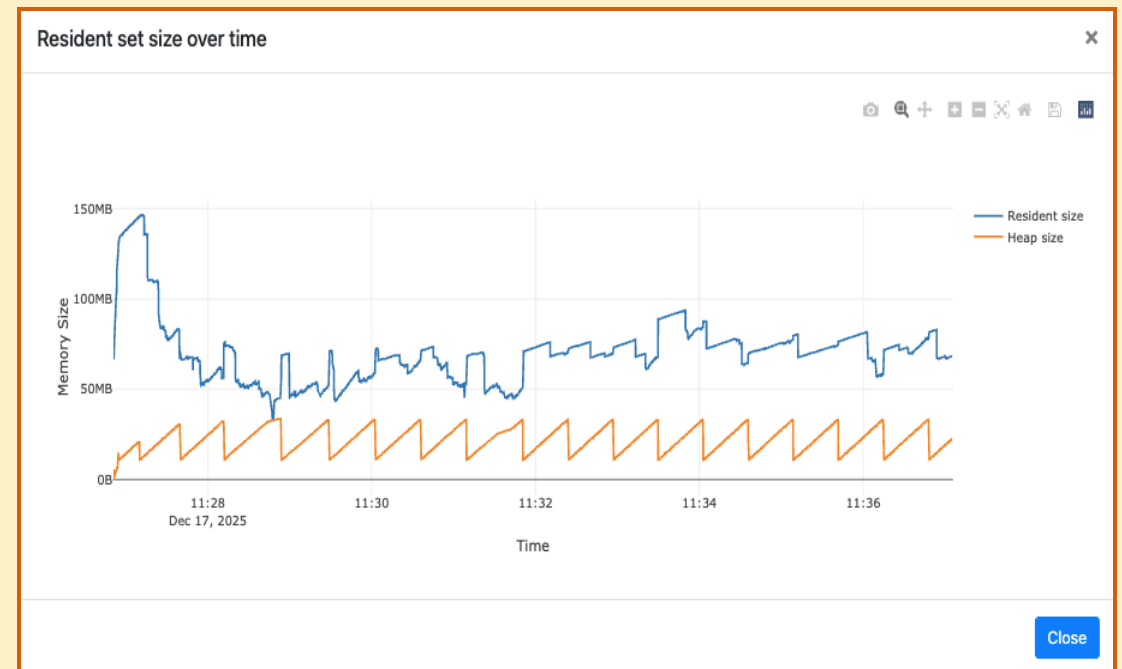
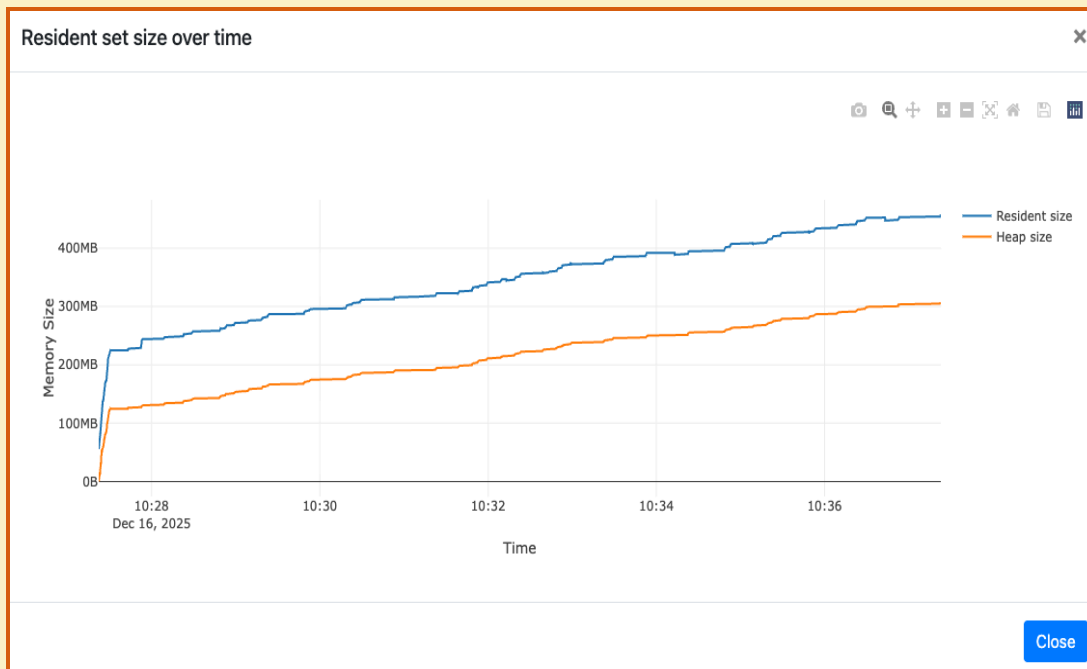
Бенчмарки ○

Backports ○

<https://github.com/python/cpython/issues/142516>

3.14

3.13

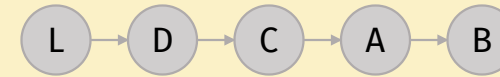


ПОЧЕМУ ПОТРЕБЛЕНИЕ ПАМЯТИ РАСТЁТ?

- Естественное состояние?
- Внутренняя утечка?
- Внешняя утечка?
- Ошибка в Python runtime/stdlib?
- Ошибка в GC?

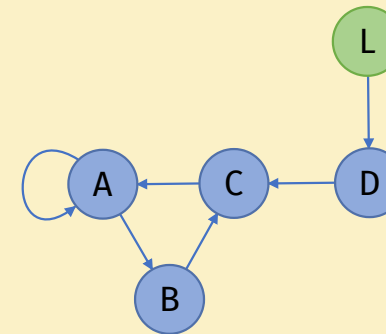
ЧТО ТАКОЕ УТЕЧКА ПАМЯТИ?

- Забыли обновить счётчик ссылок
- Не освободили неуправляемый блок памяти
- Продлили время жизни
- Сборщик мусора не разорвал циклы



GC space

object space



gc_next

gc_prev

ob_refcnt

ob_type

...

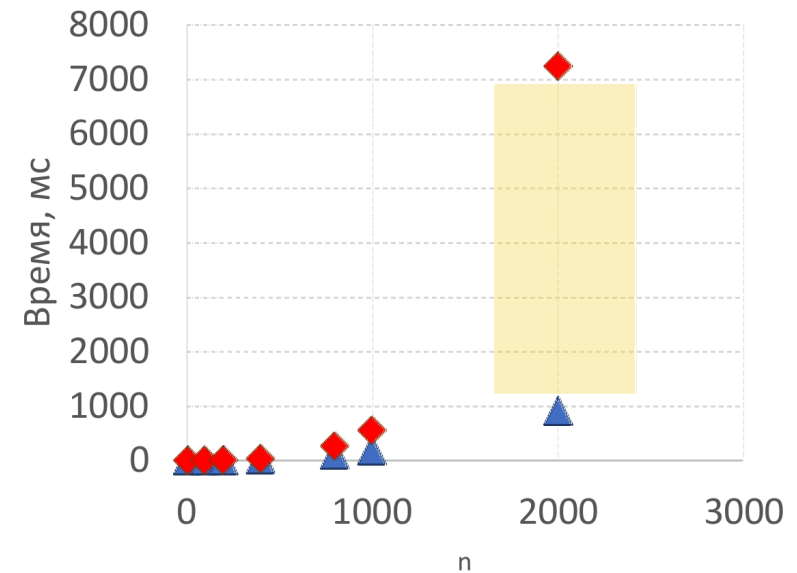
...

memory layout

ЗА ЧТО ОТВЕЧАЕТ GC?

- Определяет недостижимые объекты
- Вызывает финализаторы
- Обрабатывает слабые ссылки
- Разрывает циклы
- Удаляет объекты

```
def test(n=2000):  
    d = {}  
    for i in range(n):  
        for j in range(n):  
            d[(i,j)] = i
```



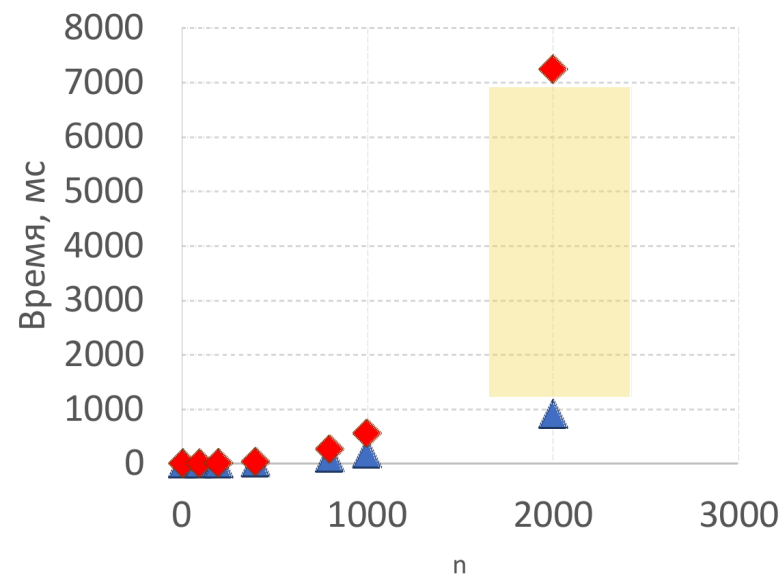
▲ 3.13 ◆ 3.14

ЗА ЧТО ОТВЕЧАЕТ GC?

- 3.14.0-3.14.4
 - Определяет живые объекты
 - Формирует инкремент
- Определяет недостижимые объекты
- Вызывает финализаторы
- Обрабатывает слабые ссылки
- Разрывает циклы
- Удаляет объекты



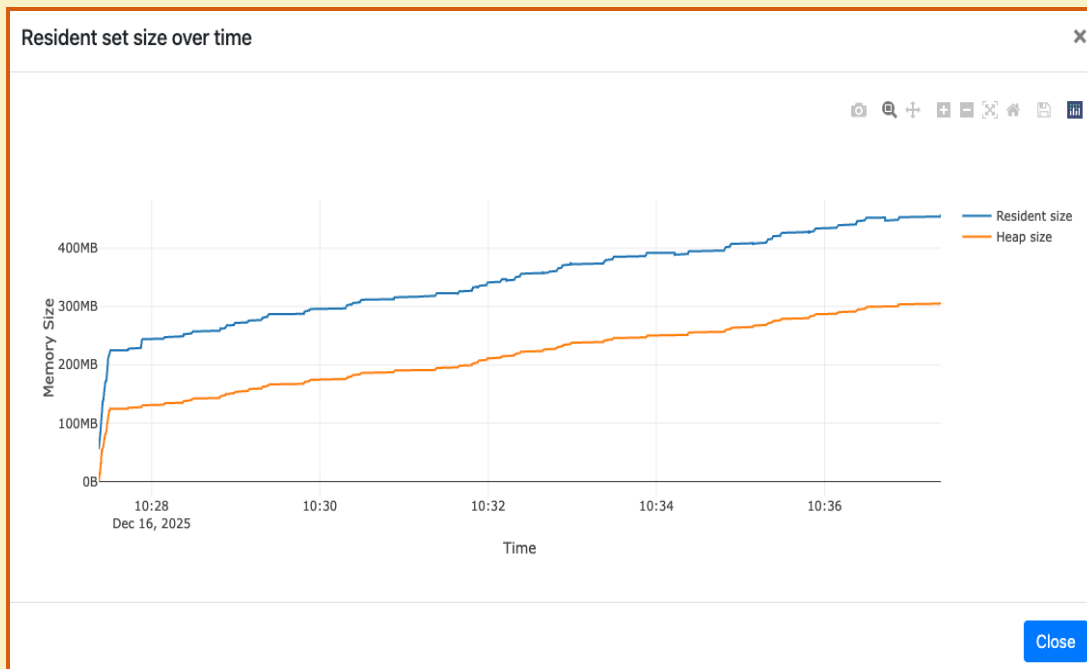
```
def test(n=2000):  
    d = {}  
    for i in range(n):  
        for j in range(n):  
            d[(i,j)] = i
```



▲ 3.13 ♦ 3.14

<https://github.com/python/cpython/issues/142516>

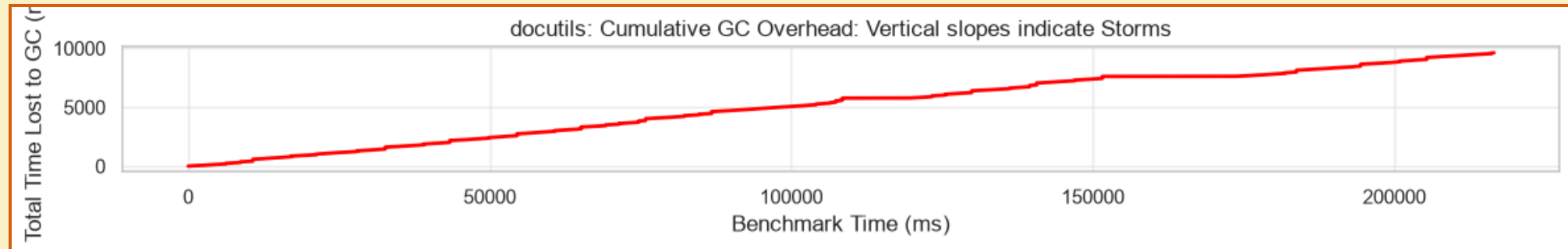
3.14



```
1 gc: collectable <Response 0x7fd8e3c4cc20>
2   → <bytes 0x561a014e9218 rc=3221225472> <b''>
3 >   → <IdentityDecoder 0x7fd8e3c4d010 rc=1> <<httpx._decoders.IdentityDecoder obje
4   → <datetime.timedelta 0x7fd8e551a490 rc=1> <datetime.timedelta(microseconds=32
5   → <int 0x561a014e71f8 rc=3221225472> <0>
6   → <Request 0x7fd8e3cf4ec0 rc=1> <<Request('HEAD', 'http://www.google.com/')>>
7 >   → <str 0x7fd8e52c9650 rc=17> <'utf-8'>
45   → <dict 0x7fd8e3c2c540 rc=1> <{'http_version': b'HTTP/1.1', 'reason_phrase': b
46   → <Headers 0x7fd8e3d6ae00 rc=1> <Headers([('content-type', 'text/html; charset
71   → <list 0x7fd8e3c2c900 rc=1> <[]>
72   → <bool 0x561a014a73c0 rc=3221225472> <True>
73   → <bool 0x561a014a73c0 rc=3221225472> <True>
74   → <NoneType 0x561a014be300 rc=3221225472> <None>
75   → <int 0x561a014e8af8 rc=3221225472> <200>
76   → <BoundAsyncStream 0x7fd8e3c4cec0 rc=1> <<httpx._client.BoundAsyncStream obje
77   → <Response 0x7fd8e3c4cc20 rc=1> <<Response [200 OK]>>
78   → <bytes 0x561a014e9218 rc=3221225472> <b''>
79 >   → <IdentityDecoder 0x7fd8e3c4d010 rc=1> <<httpx._decoders.IdentityDecoder
81   → <datetime.timedelta 0x7fd8e551a490 rc=1> <datetime.timedelta(microsecond
82   → <int 0x561a014e71f8 rc=3221225472> <0>
83 >   → <Request 0x7fd8e3cf4ec0 rc=1> <<Request('HEAD', 'http://www.google.com/'
111   → <str 0x7fd8e52c9650 rc=17> <'utf-8'>
112   → <dict 0x7fd8e3c2c540 rc=1> <{'http_version': b'HTTP/1.1', 'reason_phrase
113 >   → <Headers 0x7fd8e3d6ae00 rc=1> <Headers([('content-type', 'text/html; cha
127   → <list 0x7fd8e3c2c900 rc=1> <[]>
128   → <bool 0x561a014a73c0 rc=3221225472> <True>
129   → <bool 0x561a014a73c0 rc=3221225472> <True>
130   → <NoneType 0x561a014be300 rc=3221225472> <None>
131   → <int 0x561a014e8af8 rc=3221225472> <200>
132 >   → <BoundAsyncStream 0x7fd8e3c4cec0 rc=1> <<httpx._client.BoundAsyncStream
174   → <type 0x561a239058b0 rc=30> <<class 'httpx.Response'>>
175   → <float 0x7fd8e3cdd3b0 rc=1> <39037.104077293>
176 >   → <AsyncResponseStream 0x7fd8e3cf6120 rc=1> <<httpx._transports.default.Asyn
250   → <type 0x561a23924160 rc=15> <<class 'httpx._client.BoundAsyncStream'>>
251   → <type 0x561a239058b0 rc=30> <<class 'httpx.Response'>>
```

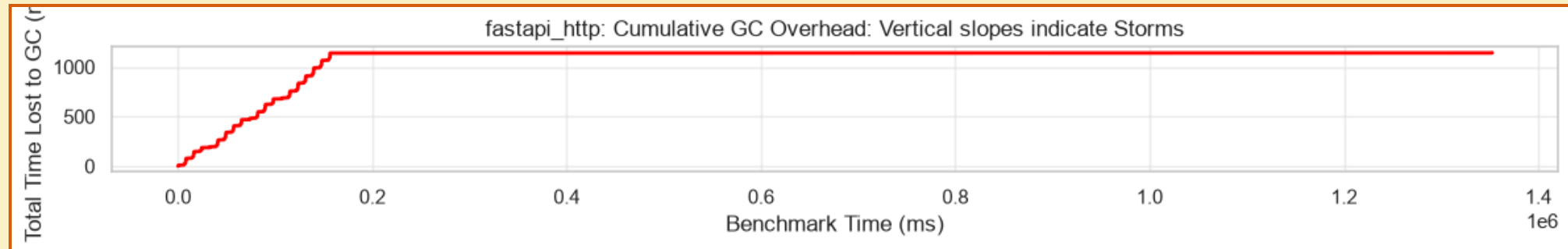
GC STORM

- Длинные/частые паузы
- Повышенное потребление памяти/CPU
- Частая полная сборка мусора



GC STORM

- Длинные/частые паузы
- Повышенное потребление памяти/CPU
- Частая полная сборка мусора



РЕЗЮМЕ

- Причины роста потребления памяти
- Что является утечкой
- Как работает подсчёт ссылок
- Как работает сборщик мусора
- Как влияет сборщик мусора на программу
- Что такое GC storm/thrashing

ИНСТРУМЕНТЫ

Проблема ●

Инструменты ●

Новое API ○

gcm on ○

Perfetto ○

Бенчмарки ○

Backports ○

ИНСТРУМЕНТЫ

- **In-process**

- `tracemalloc`
- `pympler`
- APM (ddtrace, dynatrace, ...)

- **Out-of-process**

- `memray`
- `austin`, `tachyon`
- `pyroscope`, `perforator`

- CPU
- Аллокации
- Снепшоты
- Циклы
- Время жизни
- Утечки
- Количество GC пауз
- Количество собранных объектов

МОДУЛЬ GC

- **Что не так с** `gc.callbacks`
 - Intrusive
 - C->Python interop
 - Сабинтерпретаторы
 - Heisenberg effect
- **Что не так с** `gc.get_stats`
 - Intrusive
 - When
 - Сабинтерпретаторы

НОВОЕ API

Проблема ●

Инструменты ●

Новое API ●

gcm on ○

Perfetto ○

Бенчмарки ○

Backports ○

GCMONITOR

- `_remote_debugging.get_gc_stats`
- `_remote_debugging.GCMonitor`
 - `init(pid)`
 - `get_gc_stats`

- Внешний процесс
- Стандартные метрики
 - Collections
 - Collected
 - Uncollectable
 - *Candidates*
 - *Duration*
- *Количество живых объектов*
- Время начала и окончания паузы
- Поддержка сабинтерпретаторов

GCMONITOR

```
struct pyruntimestate {
    /* This field must be first to facilitate locating it by out of process
     * debuggers. Out of process debuggers will use the offsets contained in
     * this field to be able to locate other fields in several interpreter
     * structures in a way that doesn't require them to know the exact layout
     * of those structures.
     *
     * IMPORTANT:
     * This struct is **NOT** backwards compatible between minor version of the
     * interpreter and the members, order of members and size can change
     * between minor versions. This struct is only guaranteed to be stable
     * between patch versions for a given minor version of the interpreter.
     */
    _Py_DebugOffsets debug_offsets;
    // ...
    struct pyinterpreters {
        PyMutex mutex;
        /* The linked list of interpreters, newest first. */
        PyInterpreterState *head;
        PyInterpreterState *main;
        int64_t next_id;
    } interpreters;
    // ...
};
```

```
typedef struct _Py_DebugOffsets {
    char cookie[8] _Py_NONSTRING;
    uint64_t version;
    uint64_t free_threaded;
    struct _runtime_state {
        uint64_t size;
        uint64_t finalizing;
        uint64_t interpreters_head;
    } runtime_state;
    struct _interpreter_state {
        uint64_t size;
        uint64_t id;
        uint64_t next;
        uint64_t gc;
        // ...
    } interpreter_state;
    struct _gc {
        uint64_t size;
        uint64_t collecting;
        uint64_t frame;
        uint64_t generation_stats_size;
        uint64_t generation_stats;
    } gc;
} _Py_DebugOffsets;
```

GCMONITOR

```
typedef struct _Py_DebugOffsets {
    char cookie[8] _Py_NONSTRING;
    uint64_t version;
    uint64_t free_threaded;
    uint64_t finalizing;
    struct _runtime_state {
        uint64_t size;
        uint64_t finalizing;
        uint64_t interpreters_head;
    } runtime_state;
    struct _interpreter_state {
        uint64_t size;
        uint64_t id;
        uint64_t next;
        uint64_t gc;
    } interpreter_state;
    struct _gc {
        uint64_t size;
        uint64_t collecting;
        uint64_t frame;
        uint64_t generation_stats_size;
        uint64_t generation_stats;
    } gc;
} _Py_DebugOffsets;
```

```
gscope tui —PID 24280 —Frame 122 @ 12.6s —Rate 180ms —Poll 128.6µs
_Py_DebugOffsets (embedded in _PyRuntime) @ 0x7ffbba7f1808 (size: 880 bytes)

Fields:
Py_DebugOffsets
  -- 0x0000 cookie[8] "xdebugpy"
  -- 0x0008 version 0x030f00b1
  -- 0x0010 free_threaded 0
  -- runtime_state
    -- 0x0018 size 345996
    -- 0x0020 finalizing 984
    -- 0x0028 interpreters_head 928
  -- interpreter_state
    -- 0x0030 size 226016
    -- 0x0038 id 7272
    -- 0x0040 next 7264
    -- 0x0048 threads_head 7328
    -- 0x0050 threads_main 7344
    -- 0x0058 gc 7392
    -- 0x02e8 size 176
    -- 0x02f0 collecting 112
    -- 0x02f8 frame 120
    -- 0x0300 generation_stats_size 1112
    -- 0x0308 generation_stats 104
    -- item_size 64
    -- young_entries (11) 11 x 64 = 704 bytes
    -- entry
      -- ts_start +8
      -- ts_stop +8
      -- collections +16
      -- collected +24
      -- uncollectable +32
      -- candidates +40
      -- duration +48
      -- heap_size +56
    -- index0 +784
    -- old0_entries (3) 3 x 64 bytes
    -- index1 +984
    -- old1_entries (3) 3 x 64 bytes
    -- index2 +1184

Hex Dump (DebugOffsets, 0x0000-0x036f, 880 bytes):
00000000 72 64 03 02 75 07 72 72 b1 00 0f 03 00 00 00 00 xdebugpy.....
00000010 00 00 00 00 00 00 00 00 00 44 05 00 00 00 00 00 .....D.....
00000020 00 03 00 00 00 00 00 00 00 03 00 00 00 00 00 .....h.....
00000030 e0 72 03 00 00 00 00 00 68 1c 00 00 00 00 00 .....r.....
00000040 00 1c 00 00 00 00 00 00 a0 1c 00 00 00 00 00 .....r.....
00000050 00 1c 00 00 00 00 00 00 a0 1c 00 00 00 00 00 .....r.....
00000060 a0 1d 00 00 00 00 00 00 90 1d 00 00 00 00 00 .....r.....
00000070 90 1d 00 00 00 00 00 00 10 00 00 00 00 00 .....r.....
00000080 20 1e 00 00 00 00 00 00 00 00 00 00 00 00 .....r.....
00000090 30 1e 00 00 00 00 00 00 28 1e 00 00 00 00 00 .....r.....
000000a0 50 1e 00 00 00 00 00 00 00 00 00 00 00 00 .....r.....
000000b0 68 03 00 00 00 00 00 00 00 00 00 00 00 00 .....r.....
000000c0 00 00 00 00 00 00 00 00 10 00 00 00 00 00 .....r.....
000000d0 48 00 00 00 00 00 00 00 50 00 00 00 00 00 .....r.....
000000e0 58 00 00 00 00 00 00 00 a8 00 00 00 00 00 .....r.....
000000f0 ac 00 00 00 00 00 00 00 f0 00 00 00 00 00 .....r.....
00000100 50 00 00 00 00 00 00 00 4a 00 00 00 00 00 .....r.....
00000110 28 00 00 00 00 00 00 00 80 00 00 00 00 00 .....r.....
00000120 10 01 00 00 00 00 00 00 00 00 00 00 00 00 .....r.....
00000130 58 00 00 00 00 00 00 00 08 00 00 00 00 00 .....r.....
00000140 00 00 00 00 00 00 00 00 38 00 00 00 00 00 .....r.....
00000150 50 00 00 00 00 00 00 00 4a 00 00 00 00 00 .....r.....
00000160 40 00 00 00 00 00 00 00 00 00 00 00 00 00 .....r.....
00000170 d8 00 00 00 00 00 00 00 70 00 00 00 00 00 .....r.....
00000180 78 00 00 00 00 00 00 00 80 00 00 00 00 00 .....r.....
00000190 88 00 00 00 00 00 00 00 44 00 00 00 00 00 .....r.....
000001a0 24 00 00 00 00 00 00 00 60 00 00 00 00 00 .....r.....
000001b0 68 00 00 00 00 00 00 00 00 00 00 00 00 00 .....r.....
000001c0 00 00 00 00 00 00 00 00 10 00 00 00 00 00 .....r.....
000001d0 08 00 00 00 00 00 00 00 a8 01 00 00 00 00 .....r.....
000001e0 18 00 00 00 00 00 00 00 58 00 00 00 00 00 .....r.....
000001f0 40 00 00 00 00 00 00 00 20 00 00 00 00 00 .....r.....
00000200 20 01 00 00 00 00 00 00 b0 03 00 00 00 00 .....r.....
00000210 70 03 00 00 00 00 00 00 20 00 00 00 00 00 .....r.....
00000220 20 00 00 00 00 00 00 00 10 00 00 00 00 00 .....r.....
00000230 20 00 00 00 00 00 00 00 18 00 00 00 00 00 .....r.....
00000240 10 00 00 00 00 00 00 00 c8 00 00 00 00 00 .....r.....
00000250 10 00 00 00 00 00 00 00 28 00 00 00 00 00 .....r.....
00000260 20 00 00 00 00 00 00 00 30 00 00 00 00 00 .....r.....
00000270 20 00 00 00 00 00 00 00 28 00 00 00 00 00 .....r.....
00000280 18 00 00 00 00 00 00 00 10 00 00 00 00 00 .....r.....
00000290 20 00 00 00 00 00 00 00 10 00 00 00 00 00 .....r.....
000002a0 10 00 00 00 00 00 00 00 28 00 00 00 00 00 .....r.....
000002b0 10 00 00 00 00 00 00 00 20 00 00 00 00 00 .....r.....
000002c0 40 00 00 00 00 00 00 00 20 00 00 00 00 00 .....r.....
000002d0 10 00 00 00 00 00 00 00 28 00 00 00 00 00 .....r.....
000002e0 38 00 00 00 00 00 00 00 38 00 00 00 00 00 .....r.....
000002f0 70 00 00 00 00 00 00 00 78 00 00 00 00 00 .....r.....
00000300 58 04 00 00 00 00 00 00 48 00 00 00 00 00 .....r.....
00000310 a0 00 00 00 00 00 00 00 18 00 00 00 00 00 .....r.....
00000320 48 00 00 00 00 00 00 00 43 00 00 00 00 00 .....r.....
00000330 00 00 00 00 00 00 00 00 08 00 00 00 00 00 .....r.....
00000340 18 00 00 00 00 00 00 00 48 01 00 00 00 00 .....r.....
00000350 a8 1e 00 00 00 00 00 00 80 00 00 00 00 00 .....r.....
00000360 04 00 00 00 00 00 00 00 00 02 00 00 00 00 .....r.....

PyInterpreterState @ 0x7ffbba80e128 (struct: 226016 bytes)

Key offset values stored in _Py_DebugOffsets:
  interpreter_state.id 7272
  interpreter_state.next 7264
  interpreter_state.threads_head 7328

GC struct (176 bytes) hex dump:
00000000 01 00 00 00 00 00 00 00 00 4a 8a d5 98 01 00 00 .....J.....
00000010 40 0f d7 d5 98 01 00 00 d0 07 00 00 00 03 00 00 .....r.....
00000020 28 fe 00 ba fb 7f 00 00 28 fe 00 ba fb 7f 00 00 .....r.....

[q] quit [s] save [p] pick pid [g] gc-view [t] tree [h] hex [c] collapse [d] dbg/rtr/rw [e] glitch-off [f/j/k] entry 1/17 [PageUp/Down] go
```

GC СТАТИСТИКА

```
struct gc_generation_stats {
    PyTime_t ts_start;
    PyTime_t ts_stop;
    Py_ssize_t collections;
    Py_ssize_t collected;
    Py_ssize_t uncollectable;
    Py_ssize_t candidates;
    double duration;
    Py_ssize_t heap_size;
};

struct gc_young_stats_buffer {
    struct gc_generation_stats items[11];
    int8_t index;
};

struct gc_old_stats_buffer {
    struct gc_generation_stats items[3];
    int8_t index;
};

struct gc_stats {
    struct gc_young_stats_buffer young;
    struct gc_old_stats_buffer old[2];
};
```

gcscope tui —PID 24288 —frame 766 @ 83.1s —Rate 100ms —Poll 76.1µs

GC Stats Buffer View ([g] back to full layout)

Buffer @ 0x198d51de470 (size: 1112 bytes, entry: 64 bytes)

Gen 0 (Young) — 11 entries (rate = 3.33/s, avg coll = 6.880ms)

Gen 1 (Middle) — 3 entries (rate = 0.32/s, avg coll = 5.632ms)

Gen 2 (Oldest) — 3 entries (rate = 0.8/s, avg coll = 10.781ms)

Legend: **ring** = per-generation ring index (18, points at the active entry)

gen	idx	collections	collected	heap	duration(s)
0	0	594869	1186154611	11.8K	3672.586
0	1	594870	1186156605	11.8K	3672.393
0	2	594871	1186158599	11.8K	3672.399
0	3	594872	1186160593	11.8K	3672.405
0	4	594882	1186148653	11.8K	3672.344
0	5	594863	1186142647	11.8K	3672.351
0	6	594864	1186144641	11.8K	3672.357
0	7	594865	1186146635	11.8K	3672.363
0	8	594866	1186148629	11.8K	3672.369
0	9	594867	1186150623	11.8K	3672.375
0	10	594868	1186152617	11.8K	3672.381
1	0	54078	107831532	11.8K	334.103
1	1	54076	10827544	11.8K	334.092
1	2	54077	107829538	11.8K	334.098
2	0	156	309070	11.8K	1.273
2	1	154	305882	11.8K	1.252
2	2	155	307876	11.8K	1.260

Entry #1 (gen 0, entry 0) @ 0x198d51de470

ts_start	0	+	266,436,333,253,680
ts_stop	0	+	266,436,339,072,680
collections	0	+	16 594869
collected	0	+	24 1186154611
uncollectable	0	+	32 0
candidates	0	+	40 1191522914
duration	0	+	48 3672.366371
heap_size	0	+	56 11778

[q] quit [s] save [p] pick pid [u] gc-view [t] tree [h] hex [c] collapse [d] dbg/ht [r/r] raw [g] glitch/off [t/tk] entry 1/17 [page/page] scroll

ПАУЗЫ

```
struct _PyInterpreterFrame {
    _PyStackRef f_executable;
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_globals;
    PyObject *f_builtins;
    PyObject *f_locals;
    PyFrameObject *frame_obj;
    _Py_CODEUNIT *instr_ptr;
    _PyStackRef *stackpointer;
    uint16_t return_offset;
    char owner;
    uint8_t visited;
    /* Locals and stack */
    _PyStackRef localsplus[1];
};
```

previous

frame

previous

frame

The screenshot displays the gcscope tui interface. The top bar shows 'gcscope tui —PID 24288 —Frame 766 @ 83.1s —Rate 100ms —Poll 76.1µs'. The main window is titled 'GC Stats Buffer View ([g] back to full layout)'. It contains a table with columns: gen, idx, collections, collected, heap, and duration(s). The table lists various memory segments and their statistics. Below the table, there is a section for 'Entry #1 (gen 0, entry 0) @ 0x198d51de470', which shows details for a specific memory entry, including its start, stop, collections, collected, uncollectable, candidates, duration, and heap size. The right side of the interface shows a hex dump of memory data, with each line containing a hexadecimal value and its corresponding ASCII representation.

ПОДВОДНЫЕ КАМНИ

- Не-питоновские процессы
- Дочерние процессы
- Состояние гонки
 - Не полная инициализация
 - Частичное чтение
- Повышенные права
- GC storm/thrashing

РЕЗЮМЕ

- Существующие инструменты отвечают на другие вопросы
- Использование модуля GC вносит искажения в замеры
- Новое API позволяет заглянуть внутрь GC и не вносит искажений
- Новое API не требует пауз
- Но есть подводные камни, которые надо учитывать

GCMON

Проблема ●

Инструменты ●

Новое API ●

gcmmon ●

Perfetto ○

Бенчмарки ○

Backports ○

GCMON

- **Управление процессами**

- Дочерние процессы
- Время жизни
- Метаданные
- RSS

- **Расширенные метрики**

- **Экспорт**

- Разные форматы

- Поддержка `pyperf`

- Метки

- **Оверхед**

- Rate=100ms -> 1%
- Rate=10ms -> 3-4%

- А что с gcscope?

- Perfetto Binary Format

- Chrome Trace Format

- JSONL/stdout

- OpenTelemetry?

GCMON

- **Стандартные метрики**
 - Collections
 - Collected
 - Uncollectable
 - *Candidates*
 - *Duration*
- *Количество живых объектов*
- *Время начала и окончания паузы*
- *RSS*

GCMON

- **Отладочные данные**
 - Mark Alive
 - Fill Increment
 - Deduce Unreachable
 - Handle Weakrefs Callbacks
 - Finalize Garbage
 - Handle Resurrected
 - Clear Weakrefs
 - Delete Garbage

PERFETTO

Проблема ●

Инструменты ●

Новое API ●

gcm on ●

Perfetto ●

Бенчмарки ○

Backports ○

CURRENT TRACE

pyperformance.pftrace (1 MB)

Timeline

Overview

Data Explorer

Query (SQL)

Metrics

Download

NEW TRACE

Open trace file

Open multiple trace files

Record new trace

Open Android example

Open Chrome example

SETTINGS

Settings

Flags

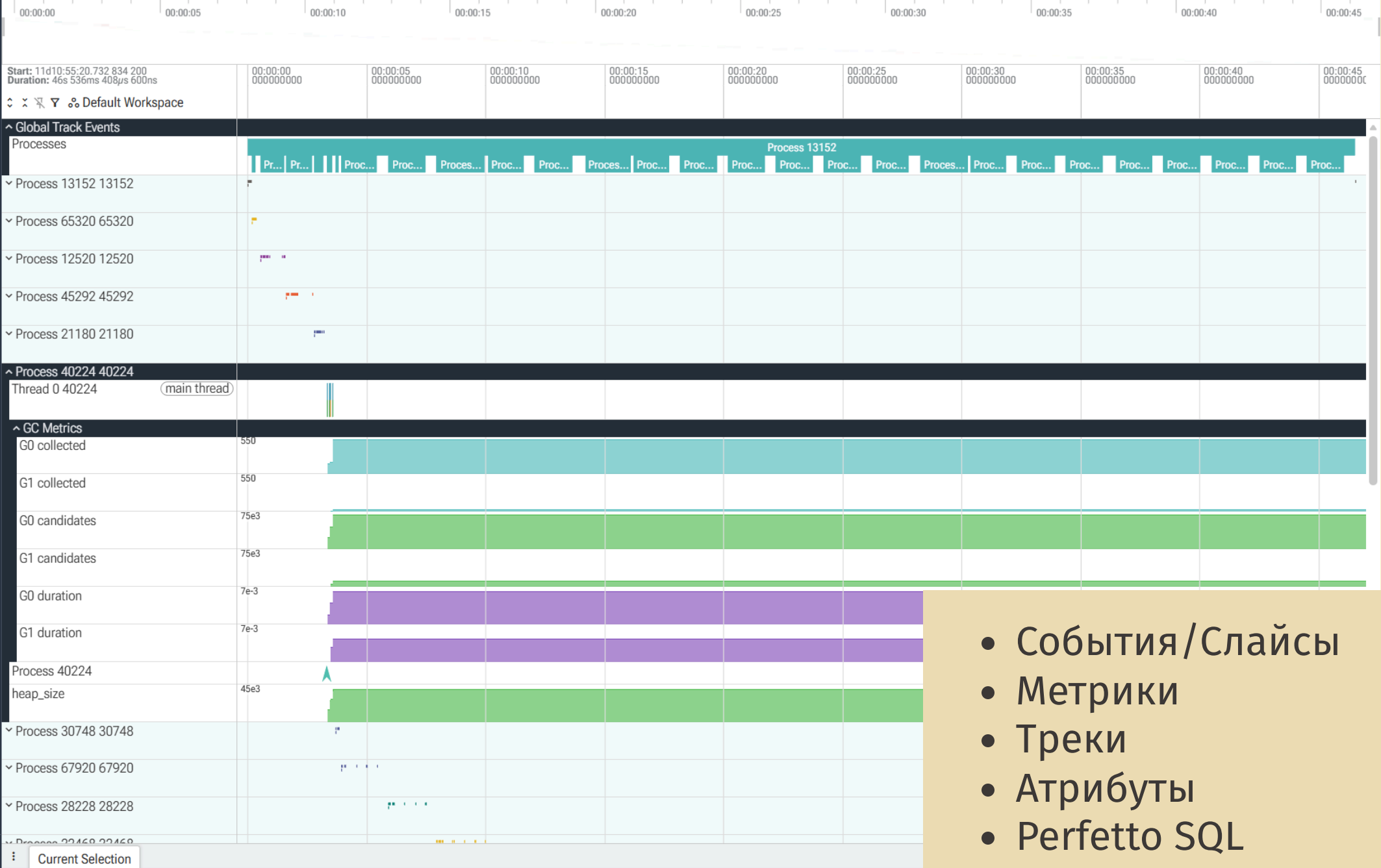
Plugins

SUPPORT

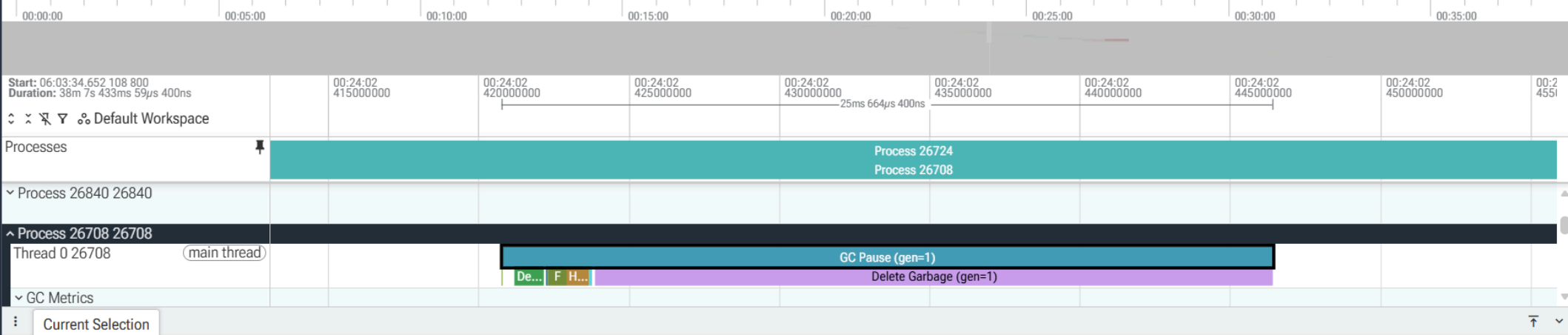
Documentation & Bugs

CONVERT TRACE

Convert to other formats



- События/Слайсы
- Метрики
- Треки
- Атрибуты
- Perfetto SQL



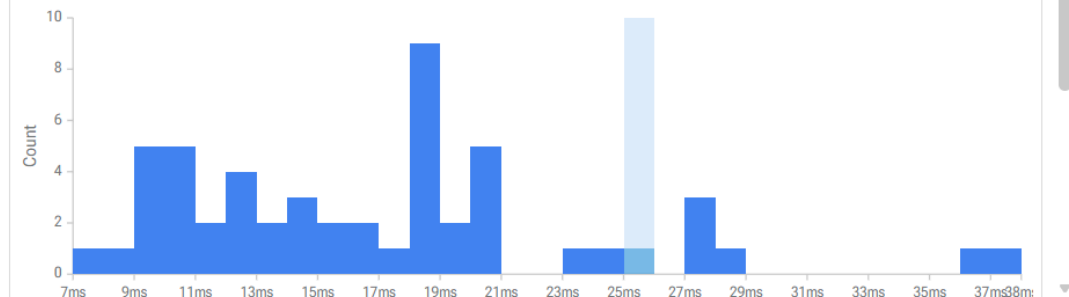
Slice GC Pause (gen=1)

Category [gc.pause\(gen=1\)](#)
Start time [00:24:02.420 760 100](#)
Duration [25ms 664µs 400ns](#)
Thread [Thread 0 \[26708\]](#)
Process [Process 26708 \[26708\]](#)
SQL ID [slice\[51794\]](#)

generation 1
iid 0
collections 22
heap_size 147289
collected 345488
uncollectable 0
candidates 456896
increment_size 8019
alive_size 37319
finalized_garbage_count 31408
deleted_garbage_count 31408
clear_weakrefs_count 0

Histogram for 'GC Pause (gen=1)'

Scope: This track



- Мини-карта процессов
- Атрибуты
- Фазы сборки мусора

CURRENT TRACE

cyclotron.pftrace (37 MB)

Timeline

Overview

Data Explorer

Query (SQL)

Metrics

Download

NEW TRACE

Open trace file

Open multiple trace files

Record new trace

Open Android example

Open Chrome example

SETTINGS

Settings

Flags

Plugins

SUPPORT

Documentation & Bugs

CONVERT TRACE

Convert to other formats

<> Query 1 +

Run Query

or press Ctrl ↵

Format

History

Tables

Query history (50 queries)

```
1 SELECT
2   name,
3   COUNT(dur) AS count,
4   ROUND(SUM(dur) / 1e6, 2) as dur_ms,
5   ROUND(AVG(dur) / 1e6, 4) AS avg_ms,
6   -- Calculate P50, P90, and P99 in milliseconds
7   ROUND(PERCENTILE(dur, 50) / 1e6, 4) AS P50_dur_ms,
8   ROUND(PERCENTILE(dur, 90) / 1e6, 4) AS P90_dur_ms,
9   ROUND(PERCENTILE(dur, 95) / 1e6, 4) AS P95_dur_ms,
10  ROUND(PERCENTILE(dur, 99) / 1e6, 4) AS P99_dur_ms
11 FROM slice
12 WHERE category IS NOT NULL
13 GROUP BY name
14 ORDER BY IIF(parent_id IS NULL, 0, 1), name
```

Returned 24 rows in 125 ms

Add debug track Export

name	count	dur_ms	avg_ms	P50_dur_ms	P90_dur_ms	P95_dur_ms	F
GC Pause (gen=0)	33746	6741.95	0.1998	0.0959	0.3035	0.4199	
GC Pause (gen=1)	1980	11499.15	5.8077	2.3585	5.9265	17.9474	
GC Pause (gen=2)	49	5074.84	103.5682	12.4377	171.8633	296.0494	
Clear Weakrefs (gen=0)	19788	72.07	0.0036	0.0029	0.0069	0.0079	
Clear Weakrefs (gen=1)	1954	91.29	0.0467	0.0472	0.0887	0.1005	
Clear Weakrefs (gen=2)	49	53.57	1.0932	0.1835	1.7276	2.2792	
Deduce Unreachable (gen=0)	33746	1964.59	0.0582	0.0467	0.0918	0.1089	
Deduce Unreachable (gen=1)	1980	1312.16	0.6627	0.5931	1.1439	1.3738	
Deduce Unreachable (gen=2)	49	167.34	3.4152	2.6387	9.3456		
Delete Garbage (gen=0)	21046	3249.07	0.1544	0.0417	0.149		
Delete Garbage (gen=1)	1952	7705.45	3.9475	0.6006	3.0049		
Delete Garbage (gen=2)	49	4229.49	86.316	2.9848	81.8673		
Fill increment (gen=1)	1978	97.9	0.0495	0.0398	0.0949		
Finalize Garbage (gen=0)	23502	490.09	0.0209	0.0059	0.0602		
Finalize Garbage (gen=1)	1954	396.85	0.2031	0.1108	0.586		

```
SELECT
name,
COUNT(dur) AS count,
ROUND(SUM(dur) / 1e6, 2) as dur_ms,
ROUND(AVG(dur) / 1e6, 4) AS avg_ms,
-- Calculate P50, P90, and P99 in
milliseconds
ROUND(PERCENTILE(dur, 50) / 1e6, 4) AS
```

```
SELECT
name,
COUNT(dur) AS count,
ROUND(SUM(dur) / 1e6, 2) as dur_ms,
ROUND(AVG(dur) / 1e6, 4) AS avg_ms,
-- Calculate P50, P90, and P99 in
milliseconds
ROUND(PERCENTILE(dur, 50) / 1e6, 4) AS
```

```
SELECT
name,
COUNT(dur) AS count,
ROUND(SUM(dur) / 1e6, 2) as dur_ms,
ROUND(AVG(dur) / 1e6, 4) AS avg_ms,
-- Calculate P50, P90, and P99 in
milliseconds
ROUND(PERCENTILE(dur, 50) / 1e6, 4) AS
```

```
SELECT
name,
COUNT(dur) AS count,
ROUND(SUM(dur) / 1e6, 2) as dur_ms,
ROUND(AVG(dur) / 1e6, 4) AS avg_ms,
-- Calculate P50, P90, and P99 in
milliseconds
ROUND(PERCENTILE(dur, 50) / 1e6, 4) AS
SELECT DISTINCT p.upid, p.pid, s.ts, s.dur
/ 1e6 as dur_ms, a.string_value as cmdline
FROM slice s
JOIN process p ON s.name = 'Process ' ||
```

- Работа с данными
- Производные метрики
- SQLite+

CURRENT TRACE

cyclotron.pftrace (37 MB)

Timeline

Overview

Data Explorer

Query (SQL)

Metrics

Download

NEW TRACE

Open trace file

Open multiple trace files

Record new trace

Open Android example

Open Chrome example

SETTINGS

Settings

Flags

Plugins

SUPPORT

Documentation & Bugs

CONVERT TRACE

Convert to other formats

<> Query 1 +

Run Query or press Ctrl ↵

Format

History

Tables

Query history (50 queries)

```
1 SELECT
2   p.ts,
3   p.dur,
4   p.name
5 FROM (SELECT * FROM slice WHERE name LIKE 'GC Pause%') p
6 INNER JOIN (SELECT * FROM slice WHERE name LIKE 'Delete Garbage%') g on p.id = g.parent_id
7 WHERE
8   (100.0 * g.dur/p.dur) > 25 AND p.dur/1e6 > 10
9
```

Returned 172 rows in 29 ms

Add debug track

Export

ts	dur	name
23085429684900	156325100	GC Pause (gen=2)
23155364252900	29611400	GC Pause (gen=2)
23159019828500	26566600	GC Pause (gen=2)
23186002205700	58527500	GC Pause (gen=2)
23255273834200	24365500	GC Pause (gen=1)
23256146116000	18237700	GC Pause (gen=1)
23256648973400	17942600	GC Pause (gen=1)
23257072868900	25664400	GC Pause (gen=1)
23257259102500	19320500	GC Pause (gen=1)
23261802915900	27232000	GC Pause (gen=1)
23262924635500	20695100	GC Pause (gen=1)
23264109573900	18039000	GC Pause (gen=1)
23267426957400	23078300	GC Pause (gen=1)
23268197362800	10299800	GC Pause (gen=1)
23269575030400	12236200	GC Pause (gen=1)
23270068817400	18046900	GC Pause (gen=1)

```
SELECT
  p.ts,
  p.dur,
  p.name
FROM (SELECT * FROM slice WHERE name LIKE
'GC Pause%') p
INNER JOIN (SELECT * FROM slice WHERE name
LIKE 'Delete Garbage%') g on p.id =

SELECT
  p.ts,
  p.dur,
  p.name
FROM (SELECT * FROM slice WHERE name LIKE
'GC Pause%') p
INNER JOIN (SELECT * FROM slice WHERE name
LIKE 'Delete Garbage%') g on p.id =

SELECT
  p.ts,
  p.dur,
  p.name
FROM (SELECT * FROM slice WHERE name LIKE
'GC Pause%') p
INNER JOIN (SELECT * FROM slice WHERE name
LIKE 'Delete Garbage%') g on p.id =

-- SELECT
-- name,
--   COUNT(dur) AS count,
--   ROUND(SUM(dur) / 1e6, 2) as dur_ms,
--   ROUND(AVG(dur) /1e6, 4) AS avg_ms,
--   -- Calculate P50, P90, and P99 in
--   milliseconds
--   ROUND(PERCENTILE(dur, 50) / 1e6, 4)
-- SELECT
-- name,
--   COUNT(dur) AS count,
--   ROUND(SUM(dur) / 1e6, 2) as dur_ms,
--   ROUND(AVG(dur) /1e6, 4) AS avg_ms,
--   -- Calculate P50, P90, and P99 in
--   milliseconds
```

- Производные метрики
- Привязка к timestamp

CURRENT TRACE

cyclotron.pftrace (37 MB)

Timeline

Overview

Data Explorer

Query (SQL)

Metrics

Download

NEW TRACE

Open trace file

Open multiple trace files

Record new trace

Open Android example

Open Chrome example

SETTINGS

Settings

Flags

Plugins

SUPPORT

Documentation & Bugs

CONVERT TRACE

Convert to other formats

Start: 06:03:34.652 108 800
Duration: 38m 7s 433ms 59µs 400ns

Default Workspace

Processes

Process 26724

GC Pause > 10ms

Delete Garbage > 10ms

Delete Garbage > 25%

Deduce Unreachable > 25%, > 10ms

Delete Garbage > 25%, > 10ms

Global Track Events

Process 26724 26724

Process 5072 5072

Process 7688 7688

Process 26472 26472

Process 16124 16124

Process 7460 7460

Process 1396 1396

Process 26840 26840

Process 26708 26708

Process 25680 25680

Process 14764 14764

Process 10788 10788

Process 25604 25604

Process 10732 10732

Process 7664 7664

Current Selection

Query Page Results

Handle Resurrected (gen=2) (across trace)

- Дополнительные треки для производных метрик

РЕЗЮМЕ

- `gsmon` — пример использования нового API
- Позволяет получить дополнительную информацию
- Использует Perfetto для хранения и визуализации данных
- Perfetto можно использовать для своих целей

БЕНЧМАРКИ

Проблема ●

Инструменты ●

Новое API ●

gcm on ●

Perfetto ●

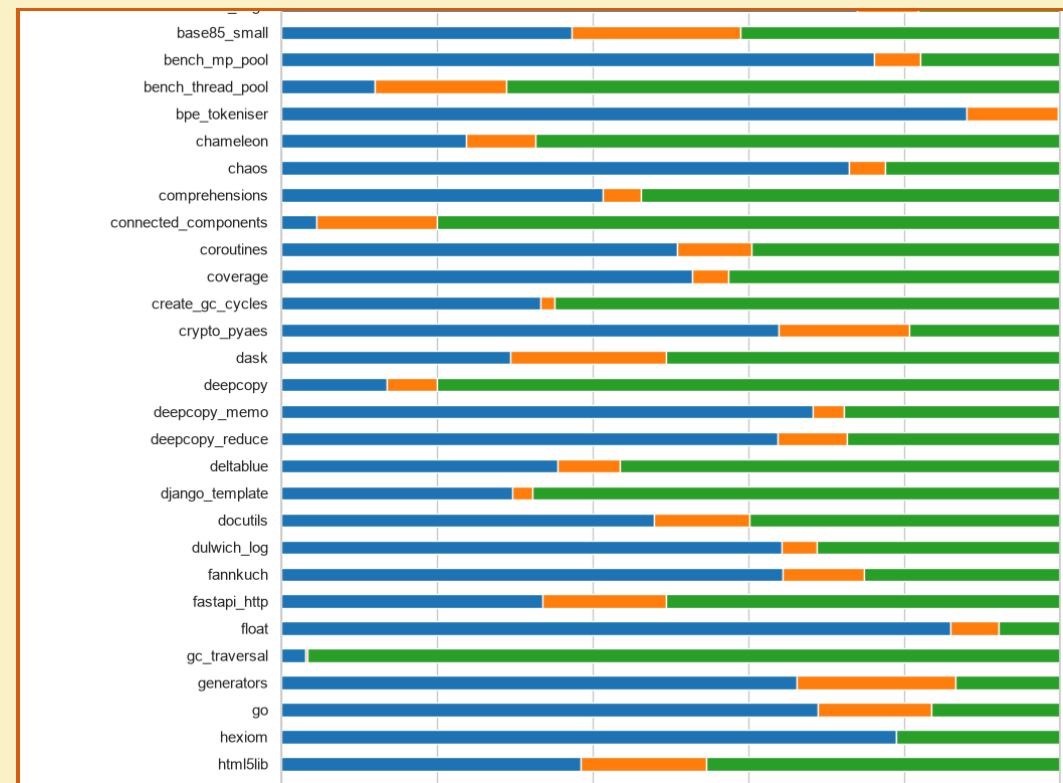
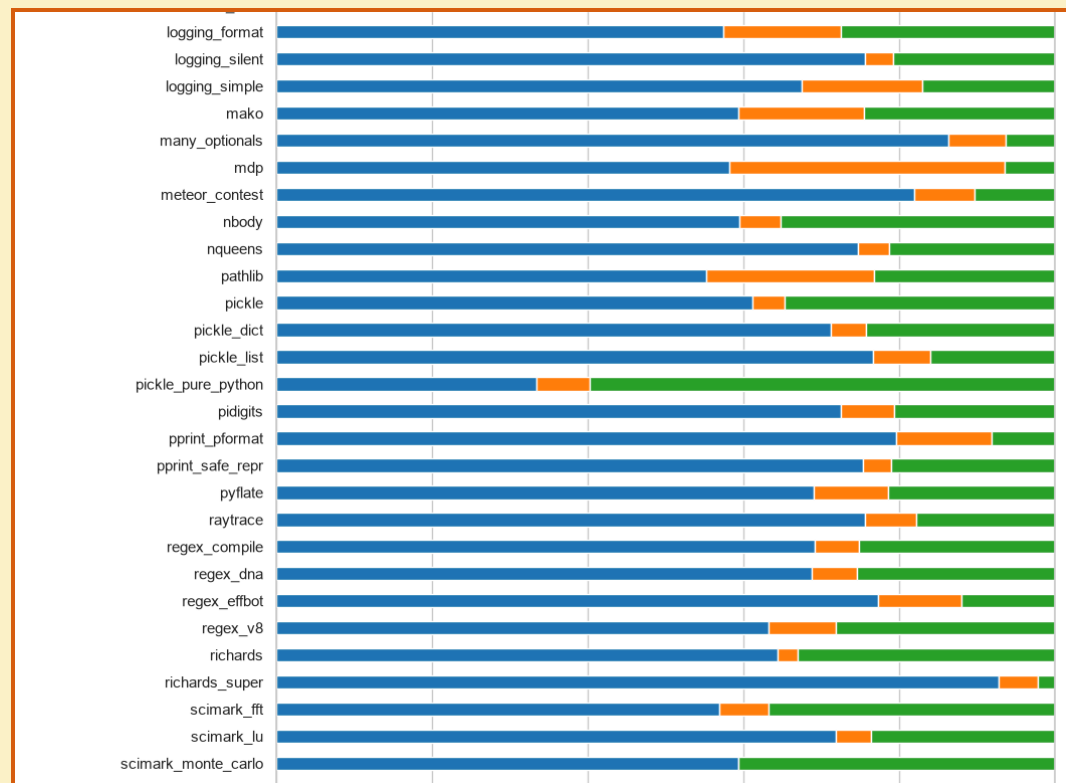
Бенчмарки ●

Backports ○

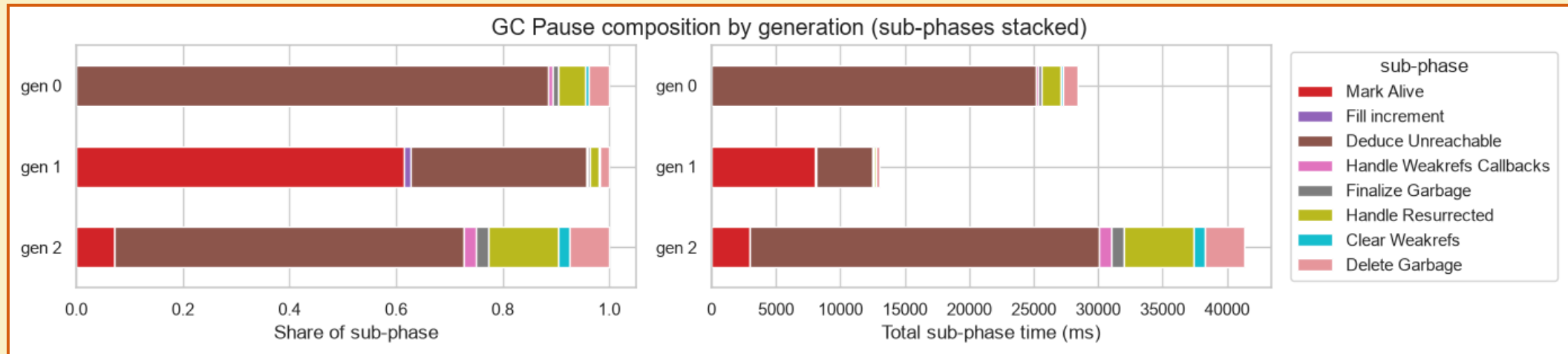
БЕНЧМАРКИ

- Почему бенчмарки?
- Pyperformance
- Cysclotron
- Any other

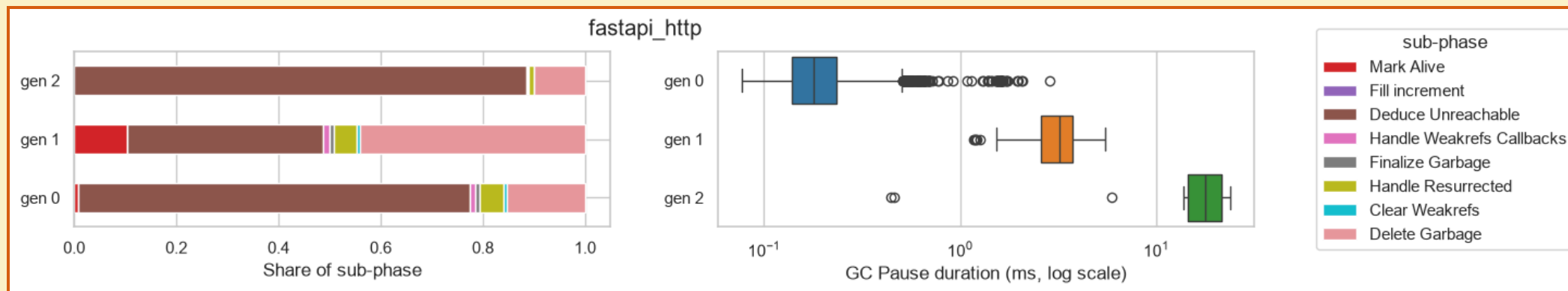
КАКОЕ ПОКОЛЕНИЕ ПРЕОБЛАДАЕТ?



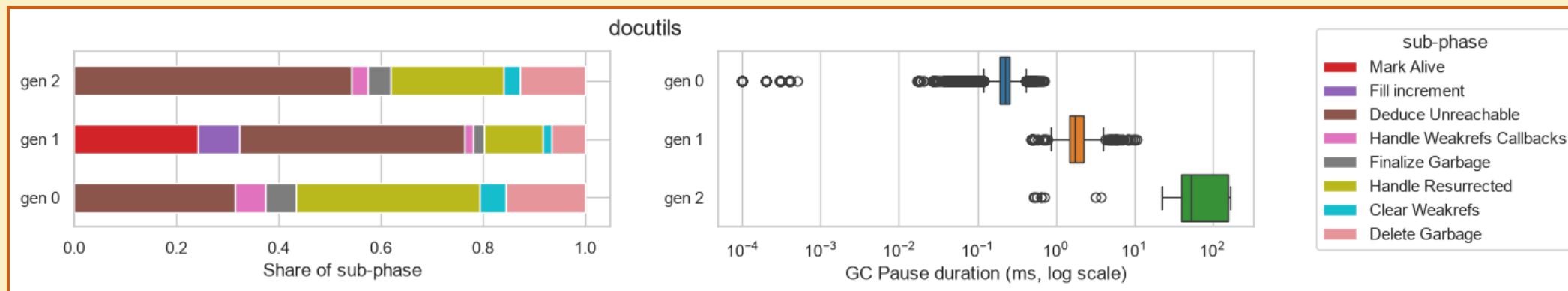
ТИПИЧНОЕ РАСПРЕДЕЛЕНИЕ ФАЗ



МНОГО ОБЪЕКТОВ С «ТЯЖЁЛЫМИ» ДАННЫМИ



ФИНАЛИЗАТОРЫ



РЕЗЮМЕ

- `gcmop` / новое API
 - можно использовать для профилирования продуктивных систем
 - можно использовать для анализа работы сборщика мусора
- Можно получить информацию о возможном тюнинге продуктивных систем

BACKPORTS

Проблема ●

Инструменты ●

Новое API ●

gcm on ●

Perfetto ●

Бенчмарки ●

Backports ●

3.13-3.14

```
typedef struct _Py_DebugOffsets {  
    char cookie[8] _Py_NONSTRING;  
    uint64_t version;  
    uint64_t free_threaded;  
    // Runtime state offset;  
    struct _runtime_state {  
        uint64_t size;  
        uint64_t finalizing;  
        uint64_t interpreters_head;  
    } runtime_state;  
    // Interpreter state offset;  
    struct _interpreter_state {  
        uint64_t size;  
        uint64_t id;  
        uint64_t next;  
        uint64_t gc;  
        // ...  
    } interpreter_state;  
    struct _gc {  
        uint64_t size;  
        uint64_t collecting;  
    } gc;  
} _Py_DebugOffsets;
```

```
/* Running stats per generation */  
struct gc_generation_stats {  
    Py_ssize_t collections;  
    Py_ssize_t collected;  
    Py_ssize_t uncollectable;  
};
```

```
struct _gc_runtime_state {  
    PyObject *trash_delete_later;  
    int trash_delete_nesting;  
    /* Is automatic collection enabled? */  
    int enabled;  
    int debug;  
    /* linked lists of container objects */  
    struct gc_generation generations[NUM_GENERATIONS];  
    PyGC_Head *generation0;  
    /* a permanent generation which won't be collected */  
    struct gc_generation permanent_generation;  
    struct gc_generation_stats generation_stats[NUM_GENERATIONS];  
    /* true if we are currently running the collector */  
    int collecting;  
    /* list of uncollectable objects */  
    PyObject *garbage;  
    /* a list of callbacks to be invoked when collection is performed */  
    PyObject *callbacks;  
};
```

3.13-3.14

```
gcscope tui —PID 4428 —Frame 220 @ 24.2s —Rate 100ms —Poll 57.8µs

_Py_DebugOffsets (embedded in _PyRuntime) @ 0x7ffbb98e3080 (size: 760 bytes)

Fields:
_Py_DebugOffsets
--- 0x0000 cookie[s] "xdebugpy"
--- 0x0008 version 0x03e01f6
--- 0x0010 free_threaded 0
--- runtime_state
| +--- 0x0018 size 315560
| +--- 0x0020 finalizing 784
| +--- 0x0022 interpreters_head 800
--- interpreter_state
+--- 0x0030 size 225632
+--- 0x0038 id 7280
+--- 0x0040 next 7272
+--- 0x0048 threads_head 7336
+--- 0x0050 threads_main 7352
+--- 0x0058 gc 7400
+--- 0x0288 size 240
+--- 0x0290 collecting 192

PyInterpreterState @ 0x7ffbb98f0f40 (struct: 225632 bytes)

Key offset values stored in _Py_DebugOffsets:
interpreter_state.id 7280
interpreter_state.next 7272
interpreter_state.threads_head 7336
interpreter_state.threads_main 7352
interpreter_state.gc 7400

GC struct (240 bytes) hex dump:
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000b0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000d0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000000f0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000100 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000120 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000130 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000170 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000180 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000190 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000001a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000001b0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000001c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000001d0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000001e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000001f0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000200 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000210 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000220 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000230 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000240 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000250 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000260 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000270 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000280 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000290 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000002a0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000002b0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000002c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000002d0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000002e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000002f0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

_Py_DebugOffsets

```
gcscope tui —PID 4428 —Frame 522 @ 57.7s —Rate 100ms —Poll 60.4µs

GC Stats Buffer View ([g] back to full layout)

Buffer @ 0x7ffbb98fac08 (size: 72 bytes, entry: 24 bytes)
Gen 0 (Young) - 1 entries (rate = n/a, avg coll = n/a)
Gen 1 (Middle) - 1 entries (rate = n/a, avg coll = n/a)
Gen 2 (Oldest) - 1 entries (rate = n/a, avg coll = n/a)

gen idx collections collected heap duration(s)
0 0 0 0 0 0.000
1 0 243154 1311020574 0 0.000
2 0 1 21 0 0.000

Entry #1 (gen 0, entry 0) @ 0x7ffbb98fac08
collections @ +0 0
collected @ +0 0
uncollectable @ +16 0
```

GC Stats Buffer

3.13-3.14

```
gcscope tui —PID 4428 —Frame 522 @ 57.7s —Rate 188ms —Poll 68.4µs
GC Stats Buffer View ([g] back to full layout)

Buffer @ 0x7ffb98faca8 (size: 72 bytes, entry: 24 bytes)
|Gen 0 (Young) - 1 entries (rate = n/a, avg coll = n/a)
|Gen 1 (Middle) - 1 entries (rate = n/a, avg coll = n/a)
|Gen 2 (Oldest) - 1 entries (rate = n/a, avg coll = n/a)
|.....|
|gen  idx  collections  collected  heap  duration(s)|
|.....|
|0      0              0          0          0          0.888|
|1      0            243154      1311828574    0          0.888|
|2      0              1          21          0          0.888|
|.....|
Entry #1 (gen 0, entry 0) @ 0x7ffb98faca8
|collections @ +0  0|
|collected @ +0  0|
|uncollectable @ +16 0|
|.....|

ts_start @ +0 266.436.533.253.680
ts_stop @ +0 266.436.539.872.680
collections @ +16 594869
collected @ +24 1186154611
uncollectable @ +32 0
candidates @ +40 1191522914
duration @ +48 3672.386371
heap_size @ +56 11778
```

3.14

```
gcscope tui —PID 24288 —Frame 766 @ 83.1s —Rate 188ms —Poll 76.1µs
GC Stats Buffer View ([g] back to full layout)

Buffer @ 0x198d51de470 (size: 1112 bytes, entry: 64 bytes)
|Gen 0 (Young) - 11 entries (rate = 3.93/s, avg coll = 6.888ms)
|Gen 1 (Middle) - 3 entries (rate = 0.32/s, avg coll = 5.632ms)
|Gen 2 (Oldest) - 3 entries (rate = 0.8/s, avg coll = 10.781ms)
|.....|
|gen  idx  collections  collected  heap  duration(s)|
|.....|
|0      0            594869      1186154611    11.8K    3672.386|
|0      1            594870      1186156605    11.8K    3672.393|
|0      2            594871      1186158599    11.8K    3672.399|
|0      3            594872      1186160593    11.8K    3672.405|
|0      4            594862      1186148653    11.8K    3672.344|
|0      5            594865      1186142647    11.8K    3672.351|
|0      6            594864      1186144641    11.8K    3672.357|
|0      7            594865      1186146635    11.8K    3672.363|
|0      8            594866      1186148629    11.8K    3672.369|
|0      9            594867      1186158623    11.8K    3672.375|
|0     10            594868      1186152617    11.8K    3672.381|
|0     11            54870      107831532    11.8K    334.103|
|1      1            54876      107827544    11.8K    334.092|
|1      2            54877      107829538    11.8K    334.098|
|2      0             156      389878    11.8K    1.273|
|2      1             154      385882    11.8K    1.252|
|2      2             155      387876    11.8K    1.268|
|.....|
Entry #1 (gen 0, entry 0) @ 0x198d51de470
|ts_start @ +0 266.436.533.253.680|
|ts_stop @ +0 266.436.539.872.680|
|collections @ +16 594869|
|collected @ +24 1186154611|
|uncollectable @ +32 0|
|candidates @ +40 1191522914|
|duration @ +48 3672.386371|
|heap_size @ +56 11778|
```

3.15+

3.8, 3.9-3.10, 3.11-3.12

- А если маркера нет, ничего нет, что делать?
- Насколько это стабильно?
- Какие факторы могут повлиять?
- Как это валидировать?



gcscope tui —PID 27784 —Frame 549 @ 59.9s —Rate 180ms —Poll 55.4µs

GC Stats Buffer View ([g] back to full layout)

Buffer @ 0x7ffb5e98378 (size: 72 bytes, entry: 24 bytes)

Gen 0 (Young) - 1 entries (rate = n/a, avg coll = n/a)

Gen 1 (Middle) - 1 entries (rate = n/a, avg coll = n/a)

Gen 2 (Oldest) - 1 entries (rate = n/a, avg coll = n/a)

gen	idx	collections	collected	heap	duration(s)
0	0	2435222	1884788558	0	0.888
1	0	221383	171358494	0	0.888
2	0	752	565794	0	0.888

Entry #1 (gen 0, entry 0) @ 0x7ffb5e98378

collections @ 0 2435222

collected @ 0 1884788558

uncollectable @ *16 0

3.8

CALL TO ACTION

- Какие метрики вам интересны и/или необходимы?
- В каких сценариях вы бы использовали эти возможности?
- Какие форматы и интеграции вам интересны?
- Какое ваше окружение?
- Какие алерты вы видите?

СПАСИБО ЗА ВНИМАНИЕ!

email: sergey.miryanov@gmail.com